// INTERNAL SHARING · 2026

Claude Code Tips

Two small but useful tips when using `claude -p` in scripts & CI/CD pipelines.

`--append-system-prompt` `costUSD` `claude -p` `--output-format json`

// QUICK CLARIFICATION · -P FLAG

What does -p mean?

📌

-P = --PRINT

Short for **--print**, not "prompt".

Runs Claude Code **non-interactively** — processes the query, prints output to stdout, then exits. No session opened.

🔄

VS INTERACTIVE MODE

Without -p: opens a persistent REPL session.

With -p: one-shot execution. Output piped to stdout. Process exits when done.

⚡

ALSO KNOWN AS

Previously called **"headless mode"** in older docs. The flag and behavior are the same — only the name changed.

💡 All CLI flags — including **--append-system-prompt** and **--output-format** — work with -p. It's the standard entry point for scripting and automation.

// TIP 01 · SYSTEM PROMPT CUSTOMIZATION

--append-system-prompt

📌

WHAT IT DOES

Appends your custom instructions to the **end** of Claude Code's default system prompt.

Does not replace — preserves all built-in capabilities.

📅

INTERACTIVE MODE SUPPORT

The flag existed before, but from **v1.0.51 (July 8, 2025)** it also works in **interactive mode**.

Non-interactive (-p) was supported earlier.

💡

OFFICIAL RECOMMENDATION

"For most use cases, use an append flag. Appending preserves Claude Code's built-in capabilities while adding your requirements."

// TIP 01 · ALL 4 SYSTEM PROMPT FLAGS

4 flags, 4 use cases

FLAG	BEHAVIOR	MODE
`--system-prompt`	**Replaces** the entire default system prompt	interactive + non-interactive ✓
`--system-prompt-file`	Replaces with contents from a file	interactive + non-interactive ✓
`--append-system-prompt`	**Appends** to the default prompt — **recommended**	interactive + non-interactive ✓
`--append-system-prompt-file`	Appends from a file	interactive + non-interactive ✓

⚠️ **`--system-prompt`** and **`--system-prompt-file`** are mutually exclusive. Append flags can be combined with either replacement flag.

📝 All 4 flags work in both interactive and non-interactive mode. Earlier docs incorrectly stated the -file variants were non-interactive only — this has since been corrected.

Two different inject mechanisms

📄

CLAUDE.MD

Injected as the **first user message** — placed after the default system prompt.

Persistent. Auto-loaded from project root or global (~/.claude/CLAUDE.md).

🔧

--APPEND-SYSTEM-PROMPT

Appended **directly into the system prompt** — not a user message.

Per-session. Must be passed on every invocation.

💡 Neither CLAUDE.md nor --append-system-prompt modifies Claude Code's default system prompt — they are two different ways to inject additional content at different positions in the context.

✅

USE CLAUDE.MD FOR

Persistent project conventions, coding standards, team rules that apply across all sessions.

⚡

USE --APPEND FOR

Task-specific behavior for a single run — CI/CD pipelines, specialized scripts, one-off instructions.

// TIP 01 · EXAMPLES

Real-world examples

```bash
# Add security focus for a single review run
claude -p "Review this auth code" \
  --append-system-prompt "Check for: SQL injection, XSS, insecure token handling."
```

```bash
# Load role instructions from a file
claude -p "$(cat prompt.txt)" \
  --append-system-prompt "$(cat ROLE.md)" \
  --permission-mode acceptEdits
```

```bash
# Interactive mode: persistent focus across the whole session
claude --append-system-prompt "Always analyze time & space complexity when writing code."
```

// TIP 01+02 · LIVE SAMPLE

JSON output walkthrough

```
claude -p "Do you know my name?" --output-format json --append-system-prompt "My name is Viet Tran"
```

```
{
  "type": "result", "subtype": "success",
  "num_turns": 1,
  "result": "Yes, your name is Viet Tran."
  "total_cost_usd": 0.07367 ← total cost for this turn
  "usage": {
    "input_tokens": 3, "output_tokens": 15,
    "cache_creation_input_tokens": 11222, ← cache write
    "cache_read_input_tokens": 6285 ← cache read
  },
  "modelUsage": { "claude-opus-4-6[1m]": {
```

- result — Claude's answer (confirms --append-system-prompt injected the name into context)
- total_cost_usd / costUSD — cost for this turn

// TIP 02 · HOW COSTUSD IS CALCULATED

How is costUSD computed?

```
input_tokens × $5 / MTok
 +  output_tokens × $25 / MTok
 +  cache_creation_input_tokens × $6.25 / MTok (5-min write)
 +  cache_read_input_tokens × $0.50 / MTok
 = costUSD
```

* Opus 4.6 pricing. Sonnet / Haiku have different rates.

Verified against the sample above (Opus 4.6):

```
3       × $5 / MTok     = $0.000015   (input)
15      × $25 / MTok    = $0.000375   (output)
11,222 × $6.25 / MTok  = $0.070138   (cache write)
6,285  × $0.50 / MTok  = $0.003143   (cache read)
```

→ Total = $0.07367 ✓ exact match with reported costUSD

// TIP 02 · WHY DOES A TINY QUERY COST SO MUCH?

Cache drives >95% of cost

🧠

CACHE WRITE — ~$0.070

11,222 tokens = Claude Code's system prompt being cached for the first time.

This is the dominant cost — but subsequent calls will read from cache at 12.5× cheaper rates.

⚡

CACHE READ — ~$0.003

6,285 tokens = context reused from a previous session's cache.

Only **$0.50/MTok** vs $5/MTok for fresh input — 90% cheaper.

💬

ACTUAL QUERY — ~$0.0004

3 input + 15 output tokens = the real question and answer.

Less than 1% of total cost — the prompt itself is almost free.

💡 **Takeaway:** Cost is front-loaded by cache writes on the first call. Subsequent calls in the same context become progressively cheaper as more tokens shift to cache reads.

// TIP 02 · IS COSTUSD ACCURATE?

Is costUSD accurate?

✅

YES — SERVER-SIDE

Computed by Anthropic **server-side**, not a client estimate.

total_cost_usd in the result message is the **authoritative value** for billing reconciliation.

📊

TOTAL_COST_USD VS COSTUSD

total_cost_usd — sum for the entire turn/session.
modelUsage[...].costUSD — per-model breakdown.

They match when only one model is used.

⚡

MULTIPLE CALLS

Each claude -p invocation returns its own costUSD. The SDK does **not** accumulate totals — sum them yourself if you need session-level tracking.

🎯 **Quick extract:** `claude -p "..." --output-format json | jq '.total_cost_usd'`

// COMBINING BOTH TIPS

Production script pattern

BASH — CI/CD

```bash
#!/bin/bash

RESULT=$(claude -p "Review PR: $(cat diff.txt)" \
  --append-system-prompt "Focus: OWASP Top 10, auth issues, data leaks." \
  --output-format json \
  --bare)

# Extract result and cost
```

--output-format json
Parse output programmatically. Extract .result and .total_cost_usd with jq.

--bare
Skips hooks, plugins, and CLAUDE.md discovery. Recommended for CI — guarantees consistent behavior on every machine.

--append-system-prompt
Add domain-specific rules without replacing Claude Code's default capabilities.

// WRAP UP

TL; DR

0️⃣ **-p = --print** (not "prompt"). Runs Claude Code non-interactively — one-shot, prints to stdout, then exits. Previously called "headless mode".

1️⃣ **--append-system-prompt** is the safe, recommended way to customize behavior per-session. All 4 system prompt flags work in both interactive and non-interactive mode.

2️⃣ **costUSD is computed server-side — 100% accurate.** Formula: input×$5 + output×$25 + cache_write×$6.25 + cache_read×$0.50 (per MTok, Opus 4.6).

3️⃣ Cache accounts for >95% of real cost. **total_cost_usd** is the authoritative field for billing. Use **--bare** in CI scripts for consistency.